

Banking Client 360 - Enterprise Architecture Blueprint

Version: 1.0
Date: October 2025
Classification: Internal - Confidential
Owner: Enterprise Architecture & Engineering

Executive Summary

This document defines the comprehensive business and technology architecture for the Banking Client 360 platform - an enterprise-grade customer relationship management system designed for retail and commercial banking operations. The platform provides a unified 360-degree view of customer relationships, accounts, transactions, and compliance data while supporting dual-database deployment (PostgreSQL and Microsoft SQL Server) and integration with Jack Henry Symitar and Silverlake core banking systems.

Key Architectural Principles:

- **Vendor Neutrality:** Database-agnostic design supporting PostgreSQL and SQL Server
 - **Security First:** PCI DSS compliant with data masking, encryption, and audit logging
 - **Regulatory Compliance:** Built-in KYC, AML, OFAC screening capabilities
 - **High Availability:** Active-active deployment with <1 hour RTO
 - **Scalability:** Horizontal scaling supporting 1M+ customers, 10K concurrent users
 - **Integration Ready:** RESTful APIs for core banking, payment processors, third-party services
-

Part 1: Business Architecture

1.1 Business Capability Model

The Banking Client 360 platform supports five macro-capability areas aligned with enterprise banking operations:

Capability Hierarchy

```
Banking Client 360 Capabilities
.
... Customer Lifecycle Management
.   ... Customer Onboarding (KYC/CDD)
.   ... Customer Profile Management
.   ... Relationship Management
.   ... Household Grouping
.   ... Customer Status Lifecycle
.
... Account & Card Services
.   ... Account Origination
.   ... Multi-Owner Account Management
.   ... Debit Card Issuance & Management
.   ... Account Status & Closure
.   ... Balance & Transaction Processing
.
... Risk & Compliance
.   ... KYC Document Management
```

- AML Transaction Monitoring
- OFAC Sanctions Screening
- Risk Rating & Assessment
- Suspicious Activity Reporting (SAR)
- .
- ... Branch Operations
 - Branch & Location Management
 - Employee Administration
 - Officer Assignment
 - Organizational Hierarchy
 - .
- ... Omni-Channel Engagement
 - Customer Search & Discovery
 - 360 Dashboard & Analytics
 - Transaction History & Reporting
 - Alert & Notification Management

Capability Ownership Matrix

Capability	Business Owner	Technical Owner	Regulatory Touchpoint
Customer Onboarding	Retail Banking VP	CRM Product Manager	BSA/AML, KYC
Account Origination	Operations Director	Core Systems Architect	Reg CC, Reg E
Debit Card Management	Card Services Manager	Payment Systems Architect	PCI DSS, Reg E
AML Monitoring	Chief Compliance Officer	Compliance Systems Lead	BSA/AML, FinCEN
OFAC Screening	Sanctions Officer	Integration Architect	OFAC, Sanctions
Risk Rating	Chief Risk Officer	Risk Analytics Lead	SR 11-7, Basel III

1.2 Business Process Models

Process 1: Customer Onboarding (Prospect-to-Client)

```

graph TB
    Start([Prospect Inquiry]) --> ProspectCreation[Create Prospect Record]
    ProspectCreation --> CustomerTypeDecision{Customer Type?}
    CustomerTypeDecision --> Individual| IndividualKYC[Collect Individual KYC]
    CustomerTypeDecision --> Business| BusinessKYC[Collect Business KYC/CDD]
    IndividualKYC --> DocumentUpload[Upload ID Documents]
    BusinessKYC --> DocumentUpload
    DocumentUpload --> OFACScreen[OFAC Screening]
    OFACScreen --> OFACMatch{Match Found?}
    OFACMatch --> Yes| Yes >80%| Reject[Reject Application]
    OFACMatch --> No| No <50%| RiskAssessment[Risk Assessment]
    OFACMatch --> Maybe| Maybe 50-80%| ManualReview[Manual Compliance Review]
  
```

```

ManualReview -->|Approved| RiskAssessment
ManualReview -->|Rejected| Reject

RiskAssessment --> AssignOfficer[Assign Relationship Officer]
AssignOfficer --> ActivateCustomer[Activate Customer]
ActivateCustomer --> GenerateCIF[Generate Jack Henry CIF]
GenerateCIF --> End([Customer Active])

Reject --> End2([Application Rejected])

```

Process Metrics:

- Target Completion Time: 24-48 hours (individual), 3-5 business days (business)
- OFAC Screening SLA: <10 seconds
- Manual Review SLA: <4 hours
- KYC Document Validation: 95% accuracy

Process 2: Account Origination

```

graph TB
    Start([Account Request]) --> ValidateCustomer{Customer Status Active?}
    ValidateCustomer -->|No| Reject[Reject Request]
    ValidateCustomer -->|Yes| AccountTypeSelection[Select Account Type]

    AccountTypeSelection --> ProductRules[Apply Product Rules]
    ProductRules --> MinDeposit{Min Deposit Met?}

    MinDeposit -->|No| RequestDeposit[Request Opening Deposit]
    MinDeposit -->|Yes| OwnershipSetup[Configure Ownership]
    RequestDeposit --> OwnershipSetup

    OwnershipSetup --> JointDecision{Joint Account?}
    JointDecision -->|Yes| AddJointOwners[Add Joint Owners<br/>Validate All Active]
    JointDecision -->|No| SingleOwner[Single Owner]

    AddJointOwners --> GenerateAccountNumber[Generate Account Number]
    SingleOwner --> GenerateAccountNumber

    GenerateAccountNumber --> CreateJHAccount[Create Jack Henry Account]
    CreateJHAccount --> CreateDebitCard{Auto-Issue Debit Card?}

    CreateDebitCard -->|Yes - Checking| IssueCard[Issue Debit Card]
    CreateDebitCard -->|No| ActivateAccount[Activate Account]
    IssueCard --> ActivateAccount

    ActivateAccount --> NotifyCustomer[Send Welcome Email]
    NotifyCustomer --> End([Account Active])

    Reject --> End2([Request Rejected])

```

Process Metrics:

- Same-Day Account Opening: 90% target
- Jack Henry Integration Response Time: <3 seconds
- Debit Card Issuance Time: <5 minutes
- Account Activation Rate: 98%

Process 3: Transaction Oversight (Real-Time Processing)

```
graph TB
    Start([Transaction Initiated]) --> TransactionType{Transaction Type}
    TransactionType -->|Debit| ValidateBalance{Sufficient Balance?}
    TransactionType -->|Credit| PostCredit[Post Credit Immediately]
    TransactionType -->|Transfer| ValidateTransfer[Validate Both Accounts]
    ValidateBalance -->|No| OverdraftCheck{Overdraft Protection?}
    ValidateBalance -->|Yes| AMLCheck[AML Threshold Check]
    OverdraftCheck -->|No| DeclineTransaction[Decline - NSF]
    OverdraftCheck -->|Yes| ChargeODFee[Charge OD Fee]
    ChargeODFee --> AMLCheck
    AMLCheck --> ThresholdExceeded{Threshold Exceeded?}
    ThresholdExceeded -->|Yes >$10K| CreateAlert[Create AML Alert]
    ThresholdExceeded -->|No| CardLimitCheck[Check Card Limits]
    CreateAlert --> CardLimitCheck
    CardLimitCheck --> LimitOK{Within Limits?}
    LimitOK -->|No| DeclineLimits[Decline - Limit Exceeded]
    LimitOK -->|Yes| PostTransaction[Post Transaction<br/>Status: Pending]
    PostTransaction --> UpdateBalance[Update Available Balance]
    UpdateBalance --> OvernightBatch[Overnight Batch Processing]
    OvernightBatch --> FinalPost[Status: Posted<br/>Update Current Balance]
    PostCredit --> FinalPost
    ValidateTransfer --> FinalPost
    FinalPost --> NotifyCustomer[Send Transaction Notification]
    NotifyCustomer --> End([Transaction Complete])
    DeclineTransaction --> End
    DeclineLimits --> End
```

Process Metrics:

- Transaction Authorization Time: <200ms

- AML Alert Generation: <1 second
- Batch Posting Window: 11 PM - 2 AM PST
- Transaction Success Rate: 99.5%

Process 4: Issue Resolution & Dispute Management

```
graph TB
    Start([Customer Complaint/Dispute]) --> CategorizeIssue{Issue Type}

    CategorizeIssue -->|Unauthorized Transaction| UnauthorizedFlow[Unauthorized Transaction Flow]
    UnauthorizedFlow --> CategorizeIssue -->|Incorrect Amount| AmountFlow[Incorrect Amount Flow]
    AmountFlow --> CategorizeIssue -->|Duplicate Charge| DuplicateFlow[Duplicate Charge Flow]
    DuplicateFlow --> CategorizeIssue -->|Other| OtherFlow[General Complaint Flow]

    UnauthorizedFlow --> TemporaryCredit{Amount >$50?}
    TemporaryCredit -->|Yes| IssueCredit[Issue Temporary Credit<br/>Within 10 Business Days]
    IssueCredit --> Investigate
    TemporaryCredit -->|No| Investigate[Investigate Claim]

    Investigate --> FraudConfirmed{Fraud Confirmed?}
    FraudConfirmed -->|Yes| PermanentCredit[Make Credit Permanent]
    PermanentCredit --> BlockCard[Block Card<br/>Issue Replacement]
    BlockCard --> End([Issue Resolved])
    FraudConfirmed -->|No| ReverseCredit[Reverse Temporary Credit]
    ReverseCredit --> NotifyCustomer1[Notify Customer - Denied]
    NotifyCustomer1 --> End

    AmountFlow --> ContactMerchant[Contact Merchant]
    DuplicateFlow --> ContactMerchant

    ContactMerchant --> MerchantResponse{Merchant Agrees?}
    MerchantResponse -->|Yes| ProcessRefund[Process Refund]
    ProcessRefund --> NotifyCustomer2[Notify Customer - Resolved]
    NotifyCustomer2 --> End
    MerchantResponse -->|No| Arbitration[Card Network Arbitration]
    Arbitration --> NotifyCustomer2

    OtherFlow --> AssignToOps[Assign to Operations]
    AssignToOps --> ResolveBranch[Branch Resolution]
    ResolveBranch --> End
```

Process Metrics:

- Initial Response Time: <4 hours
- Temporary Credit Issuance: Within 10 business days
- Final Resolution Time: 45 days maximum (Reg E)

- Customer Satisfaction: 85% target

1.3 Organizational Model

Stakeholder Hierarchy

```
graph TB
    CEO[Chief Executive Officer]

    CEO --> CFO[Chief Financial Officer]
    CEO --> CRO[Chief Risk Officer]
    CEO --> CTO[Chief Technology Officer]
    CEO --> COO[Chief Operating Officer]

    CFO --> RetailVP[VP Retail Banking]
    CFO --> CommercialVP[VP Commercial Banking]

    CRO --> ComplianceOfficer[Chief Compliance Officer]
    CRO --> RiskAnalytics[VP Risk Analytics]

    CTO --> AppDev[VP Application Development]
    CTO --> Infrastructure[VP Infrastructure]
    CTO --> Security[Chief Information Security Officer]

    COO --> BranchOps[VP Branch Operations]
    COO --> CardServices[VP Card Services]

    RetailVP --> RelationshipManagers[Relationship Managers<br/>Customer-Facing]
    CommercialVP --> BusinessBankers[Business Banking Officers<br/>Customer-Facing]

    ComplianceOfficer --> KYC_AML[KYC/AML Analysts<br/>System Users]
    ComplianceOfficer --> SanctionsOfficer[Sanctions Officer<br/>System Administrator]

    BranchOps --> BranchManagers[Branch Managers<br/>System Power Users]
    BranchOps --> Tellers[Tellers<br/>System Users]

    CardServices --> CardOps[Card Operations Specialists<br/>System Users]

    AppDev --> ProductManager[Product Manager - Client 360<br/>System Owner]
    AppDev --> DevTeam[Development Team<br/>System Builders]

    Security --> InfoSec[InfoSec Analysts<br/>Security Monitoring]
```

Role-Based Access Control (RBAC) Matrix

Role	Customer View	Account Modification	Transaction Access	Compliance Access	Admin Functions
Teller	Read-only	Limited (deposits, withdrawals)	View, limited posting	Relationship Manager	Full
Branch Manager	Full	Full	Full	View, limited actions	Branch Compliance
Compliance Officer	Full (incl. PII)	View for investigation	Full	Compliance	Risk Analyst
Risk Analyst	Read-only	Analytics	View alerts	Card Operations	Limited
Card Operations	Limited	Card-related only	Card transactions	Card management	System Administrator
System Administrator	Full	Full	Full	View audit logs	Full (read-only)
Audit Trail	Full (read-only)	View audit logs	Full (read-only)		

1.4 Business Information Architecture

Critical Data Entities & Business Meaning

Entity	Business Definition	Data Owner	Regulatory Classification	Retention Period
Customer	Individual or business entity with banking relationship	Retail/Commercial Banking	PII (Personal Identifiable Information)	7 years post-closure
Account	Financial product held by customer(s)	Operations	Financial Record	7 years post-closure
Transaction	Financial movement (debit, credit, transfer)	Operations	Financial Record	7 years
Debit Card	Payment instrument linked to checking account	Card Services	PCI DSS - Cardholder Data	7 years post-cancellation
Household	Grouping of related customers	Relationship Management	Relationship Data	Active only
Compliance Alert	KYC, AML, OFAC flag requiring investigation	Compliance	Regulatory Record	7 years minimum
Audit Log	System access and modification trail	IT Security	Security Record	7 years

Data Stewardship Rules

Customer Entity:

- **Polymorphic Model:** Individual customers use `first_name+last_name`; Business/Trust use `business_name`
- **PII Protection:** SSN/EIN displayed as last 4 digits only (*--1234)
- **Data Quality:** Database CHECK constraint enforces conditional name requirements
- **Ownership:** Customer assigned to Relationship Officer; ownership transferred via workflow

Account Entity:

- **Multi-Owner Support:** Account linked to 1-4 customers via `account_ownership` junction table
- **Primary Owner:** Designated via `is_primary_owner` flag; receives statements
- **Balance Integrity:** `current_balance` updated by transactions; `available_balance` accounts for holds
- **Status Lifecycle:** active · frozen · closed (irreversible)

Transaction Entity:

- **Timezone Consistency:** All timestamps stored and displayed in PST
- **Immutability:** Posted transactions cannot be modified (only reversed via offsetting entry)
- **Running Balance:** Calculated and stored for reconciliation
- **Categorization:** Auto-assigned via MCC mapping; customer can override

Debit Card Entity:

- **PCI Compliance:** Only `last_four_digits` stored; no full PAN/CVV/PIN
- **Account Restriction:** Cards ONLY for checking accounts (database trigger enforces)
- **Ownership Validation:** `customer_id` must exist in `account_ownership` for linked account

- **Token Storage:** jack_henry_card_id and silverlake_card_token reference core systems

1.5 Value Streams

Value Stream 1: Prospect-to-Client Conversion

Objective: Convert prospects into active banking customers within 48 hours

Value Flow:

- 1 **Marketing Lead** · Prospect record created in CRM
- 2 **Sales Contact** · Relationship Manager assigned, initial consultation
- 3 **KYC Collection** · Customer uploads ID, proof of address, tax docs
- 4 **Compliance Screening** · OFAC check, risk assessment (automated)
- 5 **Approval** · Branch Manager approval, CIF generation
- 6 **Activation** · Customer status set to "active", welcome email sent
- 7 **First Account** · Relationship Manager opens first account
- 8 **Engagement** · Customer enrolled in online banking, debit card issued

Value Metrics:

- Conversion Rate: Prospect · Active Customer = 65% target
- Cycle Time: 48 hours for individual, 5 days for business
- Customer Satisfaction: NPS >50
- Compliance Pass Rate: 98% (2% manual review)

Value Stream 2: Account Origination to First Transaction

Objective: Enable customer to transact within 24 hours of account opening

Value Flow:

- 1 **Account Request** · Customer/RM initiates account opening
- 2 **Product Selection** · Checking, savings, or business account
- 3 **Ownership Configuration** · Single or joint ownership setup
- 4 **Opening Deposit** · Minimum deposit requirement validated
- 5 **Account Creation** · Jack Henry account created, account number assigned
- 6 **Debit Card Issuance** · Card auto-issued for checking accounts
- 7 **Account Activation** · Account status set to "active"
- 8 **First Transaction** · Customer makes first deposit or purchase

Value Metrics:

- Same-Day Activation: 90% of accounts
- Debit Card Issuance: <5 minutes for checking
- First Transaction Within 24h: 75% of new accounts
- Account Opening Errors: <1%

Value Stream 3: Transaction Monitoring to AML Alert

Objective: Detect and report suspicious activity within regulatory timeframes

Value Flow:

- 1 **Transaction Posting** · Debit/credit posted to account (real-time)

- 2 **AML Threshold Check** · Automated screening for >\$10K transactions
- 3 **Pattern Detection** · Multiple transactions just below \$10K (structuring)
- 4 **Alert Generation** · AML alert created and assigned to analyst
- 5 **Investigation** · Analyst reviews customer profile, transaction history
- 6 **Risk Assessment** · Determines if SAR filing required
- 7 **SAR Filing** · Suspicious Activity Report filed with FinCEN
- 8 **Customer Action** · Account frozen or closed if confirmed fraud

Value Metrics:

- Alert Generation Time: <1 second after transaction
 - False Positive Rate: <15% (industry benchmark 20%)
 - SAR Filing Timeliness: Within 30 days (regulatory requirement)
 - Fraud Detection Rate: 85% of actual fraudulent activity
-

Part 2: Technology Architecture

2.1 System Context Diagram

```
graph TB
    subgraph "External Actors"
        Customer[Customer<br/>Web/Mobile]
        BankStaff[Bank Staff<br/>Internal Users]
        Auditor[Auditor/Regulator<br/>Read-Only Access]
    end

    subgraph "Banking Client 360 System"
        WebApp[Web Application<br/>React + Material UI]
        APIGateway[API Gateway<br/>Rate Limiting, Auth]
        AppServer[Application Server<br/>Express.js/Node.js]
        SearchEngine[Search Engine<br/>Dual-Database Adapter]
        PrimaryDB[(Primary Database<br/>PostgreSQL/SQL Server)]
        ReplicaDB[(Read Replica<br/>PostgreSQL/SQL Server)]
        CacheLayer[Redis Cache<br/>Session & Query Cache]
        AuditStore[(Audit Log Store<br/>Append-Only)]
    end

    subgraph "Core Banking Systems"
        JackHenry[Jack Henry Symitar<br/>Core Banking]
        Silverlake[Silverlake<br/>Core Banking]
    end

    subgraph "Third-Party Services"
        OFAC[OFAC API<br/>Sanctions Screening]
        CreditBureau[Credit Bureau<br/>Experian/Equifax]
        PaymentProcessor[Payment Processor<br/>Visa/Mastercard]
        DocManagement[Document Management<br/>OnBase/FileNet]
        EmailSMS[Communication Platform<br/>SendGrid/Twilio]
    end

    subgraph "Enterprise Services"
```

```

    SIEM[SIEM<br/>Splunk/QRadar]
    IdentityProvider[Identity Provider<br/>OAuth2/OIDC]
    Monitoring[Monitoring<br/>Prometheus/Grafana]
end

Customer --> WebApp
BankStaff --> WebApp
Auditor --> WebApp

WebApp --> APIGateway
APIGateway --> AppServer
AppServer --> SearchEngine
AppServer --> PrimaryDB
AppServer --> CacheLayer
SearchEngine --> PrimaryDB
SearchEngine --> ReplicaDB
AppServer --> AuditStore

AppServer -->|REST API| JackHenry
AppServer -->|REST API| Silverlake
AppServer -->|REST API| OFAC
AppServer -->|REST API| CreditBureau
AppServer -->|REST API| PaymentProcessor
AppServer -->|CMIS/S3| DocManagement
AppServer -->|SMTP/SMS| EmailSMS

APIGateway --> IdentityProvider
AppServer --> SIEM
AppServer --> Monitoring

```

External System Interfaces

System	Integration Type	Protocol	Data Flow	SLA	Details
Jack Henry Symitar	REST API	HTTPS/JSON	Bidirectional (account CRUD, transactions)	<3 sec response	
Silverlake	REST API	HTTPS/JSON	Bidirectional (account CRUD, transactions)	<3 sec response	
OFAC API	REST API	HTTPS/JSON	Outbound (name screening)	<10 sec response	
Credit Bureaus	REST API	HTTPS/JSON	Outbound (credit checks)	<5 sec response	
Payment Processors	REST API	HTTPS/JSON	Bidirectional (card authorizations)	<1 sec response	
Document Management	CMIS/S3	HTTPS	Outbound (document storage)	<2 sec upload	
Email/SMS	SMTP/REST	HTTPS	Outbound (notifications)	<10 sec delivery	
SIEM	Syslog/API	TCP/HTTPS	Outbound (audit logs)	Real-time streaming	
Identity Provider	OAuth2/OIDC	HTTPS	Inbound (authentication)	<500ms token validation	

2.2 Application Architecture

Layered Architecture Model

```

graph TB
    subgraph "Presentation Layer"
        ReactUI[React Components<br/>Material-UI + Shadcn]
        ReactRouter[Wouter Router]
        StateManagement[TanStack React Query<br/>State Management]
    end
end

```

```

subgraph "API Layer"
  APIRoutes[Express Routes<br/>/api/*]
  ValidationMiddleware[Zod Validation<br/>Request/Response]
  AuthMiddleware[Authentication<br/>JWT/Session]
  RateLimiter[Rate Limiting<br/>Throttling]
end

subgraph "Business Logic Layer"
  CustomerAdapter[Customer Adapter<br/>Business Logic]
  AccountAdapter[Account Adapter<br/>Business Logic]
  TransactionService[Transaction Service<br/>Processing
Logic]
  ComplianceService[Compliance Service<br/>AML/OFAC Logic]
  SearchService[Search Service<br/>Dual-Database Abstraction]
end

subgraph "Data Access Layer"
  DrizzleORM[Drizzle ORM<br/>Query Builder]
  SearchProviderFactory[Search Provider Factory<br/>Adapter
Pattern]
  PostgresProvider[PostgreSQL Provider<br/>Trigram Search]
  SQLServerProvider[SQL Server Provider<br/>Full-Text Search]
  CacheManager[Cache Manager<br/>Redis Operations]
end

subgraph "Integration Layer"
  JackHenryClient[Jack Henry Client<br/>Circuit Breaker]
  SilverlakeClient[Silverlake Client<br/>Circuit Breaker]
  OFACClient[OFAC Client<br/>Circuit Breaker]
  PaymentClient[Payment Gateway Client<br/>Circuit Breaker]
end

subgraph "Infrastructure Layer"
  Database[(PostgreSQL<br/>SQL Server)]
  Cache[(Redis)]
  AuditLog[(Audit Store)]
  MessageQueue[Message Queue<br/>Event Bus]
end

ReactUI --> ReactRouter
ReactRouter --> StateManagement
StateManagement --> APIRoutes

APIRoutes --> RateLimiter
RateLimiter --> AuthMiddleware
AuthMiddleware --> ValidationMiddleware
ValidationMiddleware --> CustomerAdapter
ValidationMiddleware --> AccountAdapter
ValidationMiddleware --> TransactionService
ValidationMiddleware --> ComplianceService
ValidationMiddleware --> SearchService

CustomerAdapter --> DrizzleORM

```

AccountAdapter --> DrizzleORM
TransactionService --> DrizzleORM
ComplianceService --> DrizzleORM
SearchService --> SearchProviderFactory

SearchProviderFactory --> PostgresProvider
SearchProviderFactory --> SQLServerProvider

DrizzleORM --> Database
PostgresProvider --> Database
SQLServerProvider --> Database
CacheManager --> Cache

CustomerAdapter --> JackHenryClient
AccountAdapter --> JackHenryClient
TransactionService --> JackHenryClient

CustomerAdapter --> SilverlakeClient
AccountAdapter --> SilverlakeClient

ComplianceService --> OFACClient
TransactionService --> PaymentClient

TransactionService --> MessageQueue
ComplianceService --> AuditLog

Component Responsibilities

Presentation Layer:

- **React UI Components:** Material Design-based interfaces with dense data tables
- **State Management:** React Query for server state, local state for UI
- **Client-Side Validation:** Zod schemas for form validation
- **Responsive Design:** Mobile-first approach with breakpoint-based layouts

API Layer:

- **Express Routes:** RESTful endpoints with versioning (/api/v1/*)
- **Request Validation:** Zod schema validation for all inputs
- **Authentication:** JWT token validation, session management
- **Rate Limiting:** Per-IP and per-user throttling (100 req/min)

Business Logic Layer:

- **Adapters:** Encapsulate business rules for domain entities
- **Services:** Orchestrate multi-step workflows (transactions, compliance)
- **Search Service:** Abstract dual-database fuzzy search complexity
- **Validation:** Business rule enforcement (e.g., card only on checking accounts)

Data Access Layer:

- **Drizzle ORM:** Type-safe database queries with schema validation
- **Search Provider Factory:** Runtime selection of PostgreSQL vs SQL Server provider
- **Cache Manager:** Redis caching for frequently accessed data (customer profiles, balances)

Integration Layer:

- **Circuit Breaker Pattern:** Prevent cascade failures from external systems
- **Retry Logic:** Exponential backoff for transient failures
- **Timeout Management:** 5-second default timeout per integration
- **Fallback Strategies:** Graceful degradation when external systems unavailable

2.3 Data Architecture

Database Design - Dual Vendor Strategy

Rationale for Dual-Database Support

Business Drivers:

- **Vendor Flexibility:** Avoid vendor lock-in, support customer database preferences
- **Migration Path:** Enable gradual migration from SQL Server to PostgreSQL (or vice versa)
- **Cost Optimization:** PostgreSQL for cost-sensitive deployments, SQL Server for enterprise
- **Regulatory:** Some jurisdictions prefer specific database vendors

Technical Implementation:

```
graph TB
    subgraph "Application Layer"
        App[Banking Client 360 Application]
        Config[Environment Configuration<br/>DB_VENDOR env var]
    end

    subgraph "Abstraction Layer"
        SearchFactory[SearchProviderFactory]
        ISearchProvider[ISearchProvider Interface]
    end

    subgraph "Provider Implementations"
        PostgresProvider[PostgresSearchProvider<br/>pg_trgm Trigram Similarity]
        SQLServerProvider[SqlServerSearchProvider<br/>STRING_SIMILARITY/SOUNDEX]
    end

    subgraph "Database Layer"
        PostgresDB[(PostgreSQL<br/>Primary + Replicas)]
        SQLServerDB[(SQL Server<br/>Primary + Replicas)]
    end

    subgraph "Migration Strategy"
        PostgresMigrations[PostgreSQL Migrations<br/>/migrations/postgres/]
        SQLServerMigrations[SQL Server Migrations<br/>/migrations/sqlserver/]
    end

    App --> Config
```

```
Config --> SearchFactory
SearchFactory --> ISearchProvider
ISearchProvider --> PostgresProvider
ISearchProvider --> SQLServerProvider
```

```
PostgresProvider --> PostgresDB
SQLServerProvider --> SQLServerDB
```

```
PostgresMigrations --> PostgresDB
SQLServerMigrations --> SQLServerDB
```

Schema Parity Maintenance

| Feature | PostgreSQL Implementation | SQL Server Implementation | Parity Notes |
|-----|-----|-----|-----| | **Fuzzy Search** | pg_trgm extension, trigram similarity | STRING_SIMILARITY() (2022+), SOUNDEX() (<2022) | Relevance scores within ±5% | | **Full-Text Search** | tsvector, tsquery, GIN index | Full-Text Catalogs, CONTAINS() | Different syntax, same results | | **Indexes** | GIN indexes on text fields | Full-Text indexes, computed columns | Performance parity validated | | **Triggers** | PostgreSQL trigger functions | SQL Server triggers with T-SQL | Same business logic | | **Check Constraints** | Native CHECK constraints | Native CHECK constraints | Identical validation | | **Data Types** | VARCHAR, TIMESTAMP, DECIMAL | NVARCHAR, DATETIME2, DECIMAL | Type mapping documented |

Database Schema - Core Tables

Entity Relationship Diagram (Simplified):

erDiagram

```
CUSTOMER ||--o{ ACCOUNT_OWNERSHIP : "owns"
ACCOUNT ||--o{ ACCOUNT_OWNERSHIP : "has owners"
CUSTOMER ||--o{ HOUSEHOLD_MEMBERSHIP : "member of"
HOUSEHOLD ||--o{ HOUSEHOLD_MEMBERSHIP : "contains"
ACCOUNT ||--o{ TRANSACTION : "has"
ACCOUNT ||--o{ DEBIT_CARD : "linked to"
CUSTOMER ||--o{ DEBIT_CARD : "cardholder"
DEBIT_CARD }o--|| DEBIT_CARD_LIMIT_PROFILE : "uses"
CUSTOMER ||--o{ COMPLIANCE_ALERT : "subject of"
EMPLOYEE ||--o{ CUSTOMER : "assigned to"
BRANCH ||--o{ EMPLOYEE_BRANCH : "staffed by"
EMPLOYEE ||--o{ EMPLOYEE_BRANCH : "works at"
```

```
CUSTOMER {
    bigserial customer_id PK
    varchar first_name "Conditional: individual"
    varchar last_name "Conditional: individual"
    varchar business_name "Conditional: business/trust"
    varchar full_name "Auto-generated, indexed"
    varchar tax_identifier UK "Masked display"
    varchar customer_type
    varchar customer_status
    varchar kyc_status
    varchar risk_rating
    boolean vip_customer
    boolean is_employee
}
```

```

ACCOUNT {
    bigserial account_id PK
    varchar account_number UK
    varchar account_type
    decimal current_balance
    decimal available_balance
    varchar account_status
    varchar jack_henry_account_id
}

ACCOUNT_OWNERSHIP {
    bigserial ownership_id PK
    bigint account_id FK
    bigint customer_id FK
    boolean is_primary_owner
    decimal ownership_percentage
    boolean is_active
}

DEBIT_CARD {
    bigserial card_id PK
    bigint account_id FK
    bigint customer_id FK "Database trigger validates
ownership"
    bigint limit_profile_id FK
    varchar card_type
    varchar card_status
    varchar last_four_digits "PCI: Only last 4"
    varchar card_brand
    varchar jack_henry_card_id
    varchar silverlake_card_token
}

TRANSACTION {
    bigserial transaction_id PK
    bigint account_id FK
    decimal amount
    varchar transaction_type
    varchar transaction_status
    timestamp transaction_date "PST timezone"
    timestamp posted_date "PST timezone"
    varchar category
    decimal running_balance
}

```

Data Flow Architecture

Read Path (Customer Dashboard):

```

sequenceDiagram
    participant UI as React UI
    participant API as API Gateway
    participant Cache as Redis Cache

```

```

participant App as Application Server
participant DB as Database (Primary)
participant Replica as Database (Read Replica)

UI->>API: GET /api/customers/123/dashboard
API->>Cache: Check cache key "customer:123:dashboard"

alt Cache Hit
    Cache-->>API: Return cached data
    API-->>UI: 200 OK + Dashboard data
else Cache Miss
    Cache-->>API: Cache miss
    API->>App: Process request
    App->>Replica: Query customer, accounts, transactions
    Replica-->>App: Return data
    App->>App: Calculate aggregates, format response
    App-->>Cache: Store in cache (TTL: 5 min)
    App-->>API: Dashboard data
    API-->>UI: 200 OK + Dashboard data
end

```

Write Path (Transaction Processing):

```

sequenceDiagram
    participant POS as Point of Sale
    participant Payment as Payment Processor
    participant API as API Gateway
    participant App as Application Server
    participant DB as Database (Primary)
    participant Queue as Message Queue
    participant AuditLog as Audit Log Store
    participant Replica as Database (Read Replica)

    POS->>Payment: Card authorization request
    Payment->>API: POST /api/transactions/authorize
    API->>App: Validate & process

    App->>DB: BEGIN TRANSACTION
    App->>DB: Check available balance (SELECT FOR UPDATE)

    alt Sufficient Balance
        App->>DB: INSERT transaction (status: pending)
        App->>DB: UPDATE available_balance
        App->>DB: COMMIT
        App->>Queue: Publish transaction event
        App->>AuditLog: Log transaction creation
        App-->>Payment: 200 OK - Approved
        Payment-->>POS: Authorization approved

        Note over Queue,Replica: Asynchronous Processing
        Queue->>App: Process transaction event
        App->>DB: Update transaction (status: posted)
        App->>DB: Trigger replication
        DB-->>Replica: Replicate transaction
    end

```



```

else Insufficient Balance
    App->>DB: ROLLBACK
    App->>AuditLog: Log declined transaction
    App-->>Payment: 402 Payment Required - NSF
    Payment-->>POS: Authorization declined
end

```

Database Performance & Scalability

Indexing Strategy:

Table	Index Type	Columns	Purpose	Performance Target
customer	GIN (PostgreSQL) / Full-Text (SQL Server)	first_name, last_name, business_name, full_name	Fuzzy search	<300ms for 1M records
customer	B-Tree	customer_id, tax_identifier, jack_henry_cif_number	Exact lookups	<10ms
account	B-Tree	account_id, account_number	Primary access	<10ms
account_ownership	Composite	account_id, customer_id	Join optimization	<50ms joins
transaction	B-Tree + Partial	account_id, transaction_date	WHERE status='posted'	Transaction history queries
				<100ms for 10K transactions
debit_card	B-Tree	account_id, customer_id, last_four_digits	Card lookups	<20ms

Partitioning Strategy (PostgreSQL):

```

-- Transaction table partitioned by transaction_date (monthly)
CREATE TABLE transaction_2025_10 PARTITION OF transaction
    FOR VALUES FROM ('2025-10-01') TO ('2025-11-01');
CREATE TABLE transaction_2025_11 PARTITION OF transaction
    FOR VALUES FROM ('2025-11-01') TO ('2025-12-01');
-- Automatic partition creation via cron job
-- Retention: 7 years, then archive to cold storage

```

Replication Topology:

```

graph TB
    subgraph "Production Region - US West"
        PrimaryDB["Primary Database<br/>Write/Read"]
        Replica1["Read Replica 1<br/>Read-Only"]
        Replica2["Read Replica 2<br/>Read-Only"]
        PrimaryDB --> Replica1
        PrimaryDB --> Replica2
    end

    subgraph "DR Region - US East"
        DRPrimary["DR Primary<br/>Standby"]
        DRReplica["DR Replica<br/>Standby"]
        DRPrimary --> DRReplica
    end

    PrimaryDB -->|Streaming Replication<br/>Async| Replica1
    PrimaryDB -->|Streaming Replication<br/>Async| Replica2
    PrimaryDB -->|Log Shipping<br/>Async<br/>15 min lag| DRPrimary
    DRPrimary -->|Streaming Replication| DRReplica

    subgraph "Load Balancer"
        LB["PgBouncer / SQL Server Always On"]
    end

    LB -->|Writes| PrimaryDB

```

```
LB -->|Reads| Replica1
LB -->|Reads| Replica2
```

Connection Pooling:

- **PostgreSQL:** PgBouncer with 100 max connections per app server
- **SQL Server:** Always On Availability Groups with connection pooling
- **Application Pool Size:** 20 connections per instance (10 instances = 200 total)

2.4 Integration Architecture

Integration Patterns

Pattern 1: Synchronous REST API Integration (Jack Henry / Silverlake)

```
sequenceDiagram
    participant App as Banking Client 360
    participant CB as Circuit Breaker
    participant Cache as Response Cache
    participant CoreAPI as Core Banking API<br/>(Jack Henry/Silverlake)
    participant CoreDB as Core Banking Database

    App->>CB: Request account balance
    CB->>CB: Check circuit state

    alt Circuit Open (Failure Threshold Exceeded)
        CB-->>App: 503 Service Unavailable<br/>Fallback: Cached
    balance (stale)
    else Circuit Closed (Normal)
        CB->>Cache: Check cache (TTL: 30 sec)

        alt Cache Hit
            Cache-->>CB: Return cached balance
            CB-->>App: 200 OK + Balance (cached)
        else Cache Miss
            CB->>CoreAPI: GET /accounts/{id}/balance
            CoreAPI->>CoreDB: Query balance
            CoreDB-->>CoreAPI: Balance data
            CoreAPI-->>CB: 200 OK + Balance
            CB->>Cache: Store in cache (TTL: 30 sec)
            CB-->>App: 200 OK + Balance
        end
    end

    Note over CB: Circuit state transitions:<br/>Closed · Open (5 failures in 60s)<br/>Open · Half-Open (after 30s)<br/>Half-Open · Closed (1 success)
```

Circuit Breaker Configuration:

```
const circuitBreakerConfig = {
    failureThreshold: 5,          // Open after 5 failures
```

```

        successThreshold: 2,           // Close after 2 successes in
half-open
        timeout: 5000,                 // 5 second request timeout
        resetTimeout: 30000,           // 30 seconds before retry
        fallbackFunction: getCacheBalance
    };

```

Pattern 2: Asynchronous Event-Driven Integration (Transaction Processing)

```

graph LR
    subgraph "Banking Client 360"
        App[Application Server]
        Queue[Message Queue<br/>RabbitMQ/Kafka]
        Worker[Worker Processes]
    end

    subgraph "Core Banking System"
        CoreQueue[Core Banking Queue]
        CoreProcessor[Transaction Processor]
    end

    subgraph "Third-Party Services"
        PaymentGateway[Payment Gateway]
        NotificationService[Email/SMS Service]
    end

    App -->|Publish| Queue
    Queue -->|Subscribe| Worker
    Worker -->|POST /transactions| CoreQueue
    CoreQueue --> CoreProcessor
    CoreProcessor -->|Callback| App

    Worker -->|Authorization| PaymentGateway
    Worker -->|Notification| NotificationService

```

Event Schema Example:

```

{
  "event_type": "transaction.created",
  "event_id": "evt_abc123",
  "timestamp": "2025-10-23T15:30:00-07:00",
  "payload": {
    "transaction_id": 987654,
    "account_id": 12345,
    "amount": "150.00",
    "transaction_type": "debit",
    "merchant_name": "Starbucks",
    "status": "pending"
  },
  "metadata": {
    "source": "client360_api",
    "user_id": "RM-456",
    "correlation_id": "req_xyz789"
  }
}

```

```
}
```

Pattern 3: Batch Integration (End-of-Day Processing)

```
graph TB
    subgraph "Overnight Batch Workflow"
        Start([11:00 PM PST<br/>Batch Start]) --> Extract[Extract Pending Transactions]
        Extract --> Transform[Transform & Validate]
        Transform --> Post[Post Transactions<br/>Update Balances]
        Post --> Interest[Calculate Interest]
        Interest --> Fees[Calculate Fees]
        Fees --> Reconcile[Reconcile with Core Banking]
        Reconcile --> Report[Generate Reports]
        Report --> Notify[Notify Operations]
        Notify --> End([2:00 AM PST<br/>Batch Complete])
    end

    Extract --> DB[(Database)]
    Post --> DB
    Interest --> DB
    Fees --> DB
    Reconcile --> CoreBanking[Core Banking System]
    Report --> FileServer[Report Server]
```

Batch Processing SLA:

- Start Time: 11:00 PM PST (after business hours)
- Completion Target: 2:00 AM PST (3-hour window)
- Transaction Volume: 100K transactions/day
- Reconciliation: 99.99% accuracy required

API Contract Design

RESTful API Standards:

Method	Endpoint	Purpose	Request Body	Response	Idempotent
GET	/api/v1/customers/{id}	Retrieve customer	N/A	Customer object	Yes
POST	/api/v1/customers	Create customer	Customer object + ID	Customer data	No
PUT	/api/v1/customers/{id}	Update customer (full)	Customer object	Customer data	Yes
PATCH	/api/v1/customers/{id}	Update customer (partial)	Customer object	Partial data	Yes
DELETE	/api/v1/customers/{id}	Soft delete customer	N/A	204 No Content	Yes
POST	/api/v1/customers/search	Search customers	Search criteria	Customer array + metadata	Yes

API Versioning Strategy:

- URL-based versioning: /api/v1/, /api/v2/
- Backward compatibility maintained for 2 major versions
- Deprecation notice: 6 months before version retirement
- Version header: X-API-Version: 1.0

Error Response Standard:

```
{
  "error": {
```

```

    "code": "VALIDATION_ERROR",
    "message": "Invalid customer data provided",
    "details": [
      {
        "field": "tax_identifier",
        "message": "Tax identifier must be exactly 9 digits",
        "code": "INVALID_FORMAT"
      }
    ],
    "request_id": "req_abc123",
    "timestamp": "2025-10-23T15:30:00-07:00"
  }
}

```

2.5 Security Architecture

Security Zones & Trust Boundaries

```

graph TB
    subgraph "Internet Zone - DMZ"
        WAF[Web Application Firewall<br/>ModSecurity]
        LB[Load Balancer<br/>SSL Termination]
    end

    subgraph "Application Zone - Private Network"
        API[API Gateway<br/>OAuth2 + Rate Limiting]
        AppServers[Application Servers<br/>Express.js Cluster]
    end

    subgraph "Data Zone - Highly Restricted"
        PrimaryDB[(Primary Database<br/>TDE Enabled)]
        ReplicaDB[(Read Replicas<br/>TDE Enabled)]
        Redis[(Redis<br/>Encrypted)]
    end

    subgraph "Integration Zone - Controlled"
        IntegrationGW[Integration Gateway<br/>Mutual TLS]
        CoreBanking[Core Banking Systems]
    end

    subgraph "Management Zone - Admin Only"
        Monitoring[Monitoring Systems]
        AuditLog[Audit Log Store<br/>WORM Storage]
    end

    Internet([Internet]) -->|HTTPS| WAF
    WAF -->|HTTPS| LB
    LB -->|HTTPS| API
    API -->|Internal TLS| AppServers
    AppServers -->|Encrypted Connection| PrimaryDB
    AppServers -->|Encrypted Connection| ReplicaDB
    AppServers -->|TLS| Redis

```

```

AppServers -->|Mutual TLS| IntegrationGW
IntegrationGW -->|Mutual TLS| CoreBanking
AppServers -->|Syslog TLS| AuditLog
AppServers -->|Metrics TLS| Monitoring

style "Internet Zone - DMZ" fill:#ffcccc
style "Application Zone - Private Network" fill:#fff4cc
style "Data Zone - Highly Restricted" fill:#ccffcc
style "Integration Zone - Controlled" fill:#cce5ff
style "Management Zone - Admin Only" fill:#e5ccff

```

Authentication & Authorization Architecture

Authentication Flow (OAuth 2.0 + OIDC):

```

sequenceDiagram
    participant User as Bank Employee
    participant UI as Web Application
    participant API as API Gateway
    participant IDP as Identity Provider<br/>(Okta/Azure AD)
    participant App as Application Server
    participant DB as Database

    User->>UI: Navigate to app
    UI->>IDP: Redirect to login
    User->>IDP: Enter credentials + MFA
    IDP->>IDP: Validate credentials
    IDP-->>UI: Authorization code
    UI->>IDP: Exchange code for tokens
    IDP-->>UI: Access token + ID token + Refresh token

    UI->>API: Request with Bearer token
    API->>IDP: Validate access token (JWT)
    IDP-->>API: Token valid + Claims

    API->>App: Forward request + User claims
    App->>DB: Check user roles & permissions
    DB-->>App: User roles: ["relationship_manager",
"customer_view"]
    App->>App: Authorize action (RBAC)

    alt Authorized
        App->>DB: Execute query
        DB-->>App: Data (PII masked per role)
        App-->>API: 200 OK + Data
        API-->>UI: Response
    else Unauthorized
        App-->>API: 403 Forbidden
        API-->>UI: Error response
    end
end

```

JWT Token Structure:

```
{
```

```

    "header": {
      "alg": "RS256",
      "typ": "JWT"
    },
    "payload": {
      "sub": "emp_12345",
      "name": "Jane Doe",
      "email": "jane.doe@bank.com",
      "roles": ["relationship_manager", "compliance_viewer"],
      "branch_id": "BR-001",
      "exp": 1698074400,
      "iat": 1698070800,
      "iss": "https://idp.bank.com",
      "aud": "banking-client-360"
    },
    "signature": "..."
  }
}

```

Role-Based Access Control (RBAC) Matrix:

Resource	Teller	RM	Branch Manager	Compliance	Admin
Customer - View	Branch only	Full	Full	Full (incl. PII)	
Customer - Create	·	·	·	·	·
Customer - Edit	Limited	·	·	·	·
Account - View	Branch only	Full	Full		
Account - Create	·	·	·	·	·
Account - Close	·	·	·	Requires approval	·
Transaction - View	Branch only	Full	Full	Full	Full
Transaction - Post	·	Limited	·	·	·
Compliance - View Alerts	·	·	·	View only	·
Compliance - Resolve Alerts	·	·	·	·	·
Audit Logs - View	·	·	·	·	·

Data Protection Architecture

Encryption Strategy:

Data State	Encryption Method	Key Management	Compliance
Data at Rest - Database	TDE (Transparent Data Encryption)	HSM-managed keys, rotated annually	PCI DSS, GLBA
Data at Rest - Backups	AES-256 encrypted archives	Separate backup encryption keys	PCI DSS
Data in Transit - Client to API	TLS 1.3 with perfect forward secrecy	Certificate Authority (Let's Encrypt/DigiCert)	PCI DSS
Data in Transit - API to Database	TLS 1.2+ with client certificates	Database-specific certs	PCI DSS
Data in Transit - Integrations	Mutual TLS (mTLS)	Client + Server certificates	PCI DSS
PII Fields (SSN, Tax ID)	Field-level encryption (AES-256)	Application-level key vault	PCI DSS, GDPR
Card Numbers	Tokenization (only last 4 digits stored)	Payment processor token vault	PCI DSS Level 1

Data Masking Rules:

```

// PII Masking by Role
const maskingRules = {
  tax_identifier: {
    teller: "****-**-1234", // Last 4 only
    relationship_manager: "****-**-1234",
    compliance_officer: "123-45-6789", // Full access
    auditor: "****-**-1234"
  },
  account_number: {
    teller: "*****5678", // Last 4 only
    relationship_manager: "12345678", // Full access
    compliance_officer: "12345678",
    auditor: "*****5678"
  }
}

```

```

    },
    debit_card_last_four: {
      all_roles: "1234" // Always last 4 (PCI)
    }
  };

```

PCI DSS Compliance Architecture:

```

graph TB
    subgraph "Cardholder Data Environment (CDE)"
        CardApp[Card Management Module<br/>PCI Scope]
        TokenVault[Token Vault<br/>PCI Level 1]
        CardDB[(Card Database<br/>TDE + Field Encryption)]
    end

    subgraph "Non-CDE Environment"
        CustomerApp[Customer Management<br/>Out of Scope]
        AccountApp[Account Management<br/>Out of Scope]
        MainDB[(Main Database<br/>TDE Only)]
    end

    subgraph "External PCI Services"
        PaymentProcessor[Payment Processor<br/>Visa/Mastercard]
    end

    CardApp -->|Tokenized References Only| MainDB
    CardApp -->|Full PAN| TokenVault
    TokenVault -->|Token Exchange| PaymentProcessor
    CardDB -->|Last 4 Digits Only| CardApp

    CustomerApp --> MainDB
    AccountApp --> MainDB

    style "Cardholder Data Environment (CDE)" fill:#ff9999
    style "Non-CDE Environment" fill:#99ff99

```

Key PCI DSS Controls:

- 1 **Requirement 3:** Never store full PAN, only last 4 digits
- 2 **Requirement 4:** Encrypt transmission of cardholder data
- 3 **Requirement 8:** Multi-factor authentication for card operations
- 4 **Requirement 10:** Audit all card data access
- 5 **Requirement 11:** Quarterly PCI scans and penetration testing

2.6 Infrastructure Architecture

Deployment Architecture (Kubernetes)

```

graph TB
    subgraph "Production Cluster - US West"
        subgraph "Ingress Tier"
            Ingress[NGINX Ingress Controller<br/>TLS Termination]
        end
    end

```



```

subgraph "Application Tier"
    API1[API Pod 1<br/>Node.js]
    API2[API Pod 2<br/>Node.js]
    API3[API Pod 3<br/>Node.js]
    APIService[API Service<br/>Load Balancer]
end

subgraph "Worker Tier"
    Worker1[Worker Pod 1<br/>Transaction Processing]
    Worker2[Worker Pod 2<br/>Transaction Processing]
end

subgraph "Cache Tier"
    Redis1[Redis Master]
    Redis2[Redis Replica 1]
    Redis3[Redis Replica 2]
    RedisService[Redis Service]
end

subgraph "Data Tier - External"
    PrimaryDB[(Primary Database<br/>Managed Service)]
    Replica1DB[(Read Replica 1)]
    Replica2DB[(Read Replica 2)]
end
end

Internet([Internet]) -->|HTTPS| Ingress
Ingress --> APIService
APIService --> API1
APIService --> API2
APIService --> API3

API1 --> RedisService
API2 --> RedisService
API3 --> RedisService

RedisService --> Redis1
Redis1 --> Redis2
Redis1 --> Redis3

API1 --> PrimaryDB
API2 --> PrimaryDB
API3 --> PrimaryDB

API1 --> Replica1DB
API2 --> Replica1DB
API3 --> Replica2DB

Worker1 --> PrimaryDB
Worker2 --> PrimaryDB

```

Resource Specifications:

Component	CPU	Memory	Replicas	Auto-Scaling						
API Pod	2 cores	4 GB	3 (min) - 10 (max)	CPU >70% or RPS >1000						
Worker Pod	4 cores	8 GB	2 (min) - 5 (max)							
Redis Master	2 cores	8 GB	1	N/A (stateful)						
Redis Replica	2 cores	8 GB	2	N/A (stateful)						
PostgreSQL Primary	8 cores	32 GB	1	Vertical scaling only						
PostgreSQL Replica	8 cores	32 GB	2	Add replicas horizontally						

Horizontal Pod Autoscaling (HPA) Policy:

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: banking-client-360-api
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Pods
      pods:
        metric:
          name: requests_per_second
        target:
          type: AverageValue
          averageValue: "1000"

```

High Availability & Disaster Recovery

RTO/RPO Targets:

Tier	RTO (Recovery Time Objective)	RPO (Recovery Point Objective)	Strategy
Tier 1 - Critical (Customer Transactions)	1 hour	15 minutes	Active-Active multi-region
Tier 2 - Important (Customer Dashboard)	4 hours	1 hour	Active-Passive with replication
Tier 3 - Standard (Reporting)	24 hours	24 hours	Backup restore

Disaster Recovery Architecture:

```

graph TB
    subgraph "Primary Region - US West (Oregon)"
        PrimaryLB[Load Balancer]
        PrimaryApp[App Cluster<br/>3-10 pods]
        PrimaryDB[(Primary DB<br/>Write/Read)]
        PrimaryReplica[(Replica DB<br/>Read-Only)]
    end

    subgraph "DR Region - US East (Virginia)"
        DRLB[Load Balancer<br/>Standby]
        DRApp[App Cluster<br/>Standby]
    end

```

```

        DRDB[(DR DB<br/>Standby)]
    end

    subgraph "Global Traffic Manager"
        GTM[Azure Traffic Manager<br/>or AWS Route 53]
    end

    GTM -->|Active| PrimaryLB
    GTM -->|Failover| DRLB

    PrimaryLB --> PrimaryApp
    PrimaryApp --> PrimaryDB
    PrimaryApp --> PrimaryReplica

    DRLB --> DRApp
    DRApp --> DRDB

    PrimaryDB -->|Log Shipping<br/>15 min lag| DRDB

    PrimaryDB -->|Streaming<br/>Real-time| PrimaryReplica

```

Failover Procedure:

- 1 **Detection:** Health check fails 3 consecutive times (90 seconds)
- 2 **Alert:** PagerDuty alert to on-call engineer + auto-escalation
- 3 **Validation:** Automated validation script confirms primary region down
- 4 **DNS Failover:** Traffic Manager redirects to DR region (TTL: 60 seconds)
- 5 **Database Promotion:** DR database promoted to primary (manual approval required)
- 6 **Application Startup:** DR app cluster scales up to production capacity
- 7 **Verification:** Smoke tests confirm DR region operational
- 8 **Communication:** Status page updated, customer notification sent

Estimated Failover Time: 45-60 minutes (within 1-hour RTO)

Monitoring & Observability

Observability Stack:

```

graph TB
    subgraph "Application Layer"
        App[Banking Client 360<br/>Application]
    end

    subgraph "Metrics & Monitoring"
        Prometheus[Prometheus<br/>Metrics Collection]
        Grafana[Grafana<br/>Dashboards]
        AlertManager[Alert Manager<br/>Alert Routing]
    end

    subgraph "Logging"
        Fluentd[Fluentd<br/>Log Aggregation]
        Elasticsearch[Elasticsearch<br/>Log Storage & Search]
        Kibana[Kibana<br/>Log Analysis]
    end

```

```

subgraph "Tracing"
    OTel[OpenTelemetry<br/>Trace Collector]
    Jaeger[Jaeger<br/>Distributed Tracing]
end

subgraph "External SIEM"
    Splunk[Splunk<br/>Security Analytics]
end

App -->|Metrics| Prometheus
App -->|Logs| Fluentd
App -->|Traces| OTel

Prometheus --> Grafana
Prometheus --> AlertManager

Fluentd --> Elasticsearch
Fluentd --> Splunk
Elasticsearch --> Kibana

OTel --> Jaeger

```

Key Metrics & Alerts:

Metric	Threshold	Alert Severity	Action
API Response Time (P95)	>1 second	Warning	Review slow queries
API Response Time (P99)	>3 seconds	Critical	Immediate investigation
Error Rate	>1%	Warning	Check logs
Error Rate	>5%	Critical	Page on-call engineer
Database Connection Pool	>80% utilization	Warning	Scale up connections
CPU Utilization	>80%	Warning	Trigger auto-scaling
Memory Utilization	>85%	Critical	Immediate scaling
Disk Space	<20% free	Warning	Cleanup or expand
Transaction Processing Lag	>5 minutes	Critical	Check worker health

Dashboard Examples:

- Executive Dashboard** (Real-time KPIs)
 - Active users (current)
 - Transactions per minute
 - System availability (SLA: 99.9%)
 - Revenue impact (transactions * avg value)
- Operations Dashboard** (Health Metrics)
 - API response times (P50, P95, P99)
 - Error rates by endpoint
 - Database query performance
 - Cache hit rates
- Security Dashboard** (Threat Detection)
 - Failed login attempts
 - API rate limit violations
 - Suspicious transaction patterns
 - OFAC/AML alert volume

Part 3: Enterprise Patterns & Design

3.1 Adapter Pattern for Dual-Database Support

Intent: Abstract database-specific search logic to support multiple database vendors transparently.

Problem:

- PostgreSQL and SQL Server have different fuzzy search capabilities
- Need consistent search results regardless of database backend
- Avoid vendor lock-in while maintaining performance

Solution:

```
// Interface defining search contract
interface ISearchProvider {
  searchCustomers(criteria: SearchCriteria):
  Promise<CustomerSearchResult[]>;
  searchAccounts(criteria: SearchCriteria):
  Promise<AccountSearchResult[]>;
}

// PostgreSQL implementation
class PostgresSearchProvider implements ISearchProvider {
  async searchCustomers(criteria: SearchCriteria):
  Promise<CustomerSearchResult[]> {
    // Use pg_trgm trigram similarity
    const query = sql`
      SELECT
        customer_id,
        full_name,
        similarity(full_name, ${criteria.searchTerm}) as
relevance_score
      FROM customer
      WHERE similarity(full_name, ${criteria.searchTerm}) >
${criteria.threshold}
      ORDER BY relevance_score DESC
      LIMIT ${criteria.limit}
    `;
    return db.execute(query);
  }
}

// SQL Server implementation
class SqlServerSearchProvider implements ISearchProvider {
  async searchCustomers(criteria: SearchCriteria):
  Promise<CustomerSearchResult[]> {
    // Use STRING_SIMILARITY (SQL Server 2022+) or SOUNDEX fallback
    if (this.version >= 2022) {
      const query = sql`
        SELECT
          customer_id,
          full_name,
          STRING_SIMILARITY(full_name, ${criteria.searchTerm}) as
relevance_score
        FROM customer
        WHERE STRING_SIMILARITY(full_name, ${criteria.searchTerm})
> ${criteria.threshold}
        ORDER BY relevance_score DESC
      `;
      return db.execute(query);
    } else {
      // Fallback to SOUNDEX for older versions
      const query = sql`
        SELECT
          customer_id,
          full_name,
          SOUNDEX(full_name) as
relevance_score
        FROM customer
        WHERE SOUNDEX(full_name) > ${criteria.threshold}
        ORDER BY relevance_score DESC
      `;
      return db.execute(query);
    }
  }
}
```

```

        OFFSET 0 ROWS FETCH NEXT ${criteria.limit} ROWS ONLY
    `;
    return db.execute(query);
} else {
    // Fallback to SOUNDEX for older versions
    const query = sql`
        SELECT
            customer_id,
            full_name,
            DIFFERENCE(full_name, ${criteria.searchTerm}) as
relevance_score
        FROM customer
        WHERE DIFFERENCE(full_name, ${criteria.searchTerm}) >= 3
        ORDER BY relevance_score DESC
    `;
    return db.execute(query);
}
}
}
// Factory for runtime provider selection
class SearchProviderFactory {
    static createProvider(): ISearchProvider {
        const dbVendor = process.env.DB_VENDOR || 'postgres';

        switch (dbVendor) {
            case 'postgres':
                return new PostgresSearchProvider();
            case 'sqlserver':
                return new SqlServerSearchProvider();
            default:
                throw new Error(`Unsupported database vendor:
${dbVendor}`);
        }
    }
}
// Usage in application
const searchProvider = SearchProviderFactory.createProvider();
const results = await searchProvider.searchCustomers({ searchTerm:
'Smith', threshold: 0.3, limit: 50 });

```

Benefits:

- Single codebase supports multiple databases
- Zero code changes to switch database vendors
- Consistent API for search operations
- Easy to add new database providers

3.2 Circuit Breaker Pattern for Integration Resilience

Intent: Prevent cascade failures when external systems (core banking, payment processors) are unavailable.

Problem:

- Core banking system downtime should not crash the entire application

- Repeated failed requests waste resources and delay failure detection
- Need graceful degradation with fallback strategies

Solution:

```
class CircuitBreaker {
  private state: 'CLOSED' | 'OPEN' | 'HALF_OPEN' = 'CLOSED';
  private failureCount: number = 0;
  private lastFailureTime: number = 0;

  constructor(
    private failureThreshold: number = 5,
    private resetTimeout: number = 30000, // 30 seconds
    private requestTimeout: number = 5000, // 5 seconds
    private fallbackFunction?: () => any
  ) {}

  async execute(fn: () => Promise<any>): Promise<any> {
    if (this.state === 'OPEN') {
      if (Date.now() - this.lastFailureTime > this.resetTimeout) {
        this.state = 'HALF_OPEN';
        console.log('[CircuitBreaker] Attempting to close circuit (HALF_OPEN)');
      } else {
        console.log('[CircuitBreaker] Circuit OPEN, returning fallback');
        if (this.fallbackFunction) {
          return this.fallbackFunction();
        }
        throw new Error('Service unavailable - circuit breaker open');
      }
    }

    try {
      const result = await this.withTimeout(fn(),
this.requestTimeout);
      this.onSuccess();
      return result;
    } catch (error) {
      this.onFailure();
      throw error;
    }
  }

  private async withTimeout(promise: Promise<any>, timeout:
number): Promise<any> {
    return Promise.race([
      promise,
      new Promise((_, reject) =>
        setTimeout(() => reject(new Error('Request timeout')),
timeout)
      )
    ]);
  }
}
```

```

    }

    private onSuccess() {
      this.failureCount = 0;
      if (this.state === 'HALF_OPEN') {
        this.state = 'CLOSED';
        console.log('[CircuitBreaker] Circuit CLOSED');
      }
    }

    private onFailure() {
      this.failureCount++;
      this.lastFailureTime = Date.now();

      if (this.failureCount >= this.failureThreshold) {
        this.state = 'OPEN';
        console.log(`[CircuitBreaker] Circuit OPEN after
${this.failureCount} failures`);
      }
    }
  }
}
// Usage with Jack Henry integration
class JackHenryClient {
  private circuitBreaker: CircuitBreaker;

  constructor() {
    this.circuitBreaker = new CircuitBreaker(
      5, // Open after 5 failures
      30000, // 30 second reset timeout
      5000, // 5 second request timeout
      this.getCachedAccountBalance // Fallback function
    );
  }

  async getAccountBalance(accountId: number): Promise<number> {
    return this.circuitBreaker.execute(async () => {
      const response = await
fetch(`${JH_API_URL}/accounts/${accountId}/balance`, {
  headers: { 'Authorization': `Bearer ${this.token}` }
});

      if (!response.ok) {
        throw new Error(`Jack Henry API error:
${response.status}`);
      }

      const data = await response.json();
      this.cacheAccountBalance(accountId, data.balance); // Cache
for fallback
      return data.balance;
    });
  }

  private getCachedAccountBalance(accountId: number): number {

```



```

    // Return stale cached data when Jack Henry is down
    const cached = this.cache.get(`account:${accountId}:balance`);
    if (cached) {
        console.log(`[Fallback] Returning cached balance for account
${accountId}`);
        return cached;
    }
    throw new Error('No cached data available');
}
}

```

3.3 CQRS (Command Query Responsibility Segregation) Pattern

Intent: Separate read and write operations for optimal performance and scalability.

Problem:

- Customer dashboard requires complex joins across multiple tables (slow)
- Transaction processing requires fast writes with minimal locking
- Read and write workloads have different scaling characteristics

Solution:

```

graph TB
    subgraph "Command Side (Writes)"
        CommandAPI[Command API<br/>POST/PUT/DELETE]
        CommandHandler[Command Handlers]
        WriteDB[(Primary Database<br/>Normalized Schema)]
        EventBus[Event Bus]
    end

    subgraph "Query Side (Reads)"
        QueryAPI[Query API<br/>GET]
        QueryHandler[Query Handlers]
        ReadDB[(Read Models<br/>Denormalized Views)]
        Cache[Redis Cache]
    end

    CommandAPI --> CommandHandler
    CommandHandler --> WriteDB
    CommandHandler --> EventBus

    EventBus --> QueryHandler
    QueryHandler --> ReadDB
    QueryHandler --> Cache

    QueryAPI --> Cache
    Cache -->|Cache Miss| ReadDB

    WriteDB -->|Replication| ReadDB

```

Implementation Example:

```

// Command Side: Create customer
class CreateCustomerCommand {

```

```

    constructor(
      public firstName: string,
      public lastName: string,
      public taxIdentifier: string
    ) {}
  }
}
class CreateCustomerHandler {
  async execute(command: CreateCustomerCommand): Promise<number> {
    // Write to normalized database
    const customerId = await db.insert(customer).values({
      firstName: command.firstName,
      lastName: command.lastName,
      taxIdentifier: command.taxIdentifier,
      customerStatus: 'active'
    });

    // Publish event for read model update
    await eventBus.publish({
      type: 'CustomerCreated',
      payload: { customerId, ...command },
      timestamp: new Date()
    });

    return customerId;
  }
}
// Query Side: Customer dashboard
class CustomerDashboardQuery {
  constructor(public customerId: number) {}
}
class CustomerDashboardHandler {
  async execute(query: CustomerDashboardQuery):
Promise<CustomerDashboard> {
    // Check cache first
    const cacheKey = `customer:${query.customerId}:dashboard`;
    const cached = await cache.get(cacheKey);
    if (cached) return JSON.parse(cached);

    // Query denormalized read model
    const dashboard = await db.select({
      customerId: customerDashboardView.customerId,
      fullName: customerDashboardView.fullName,
      totalBalance: customerDashboardView.totalBalance,
      accountCount: customerDashboardView.accountCount,
      lastTransactionDate:
customerDashboardView.lastTransactionDate,
      riskRating: customerDashboardView.riskRating
    })
    .from(customerDashboardView)
    .where(eq(customerDashboardView.customerId, query.customerId))
    .limit(1);

    // Cache for 5 minutes
    await cache.set(cacheKey, JSON.stringify(dashboard), 'EX',

```

```

300);

    return dashboard;
}
}
// Read model materialized view (PostgreSQL)
CREATE MATERIALIZED VIEW customer_dashboard_view AS
SELECT
    c.customer_id,
    c.full_name,
    c.risk_rating,
    COALESCE(SUM(a.current_balance), 0) as total_balance,
    COUNT(DISTINCT a.account_id) as account_count,
    MAX(t.transaction_date) as last_transaction_date
FROM customer c
LEFT JOIN account_ownership ao ON c.customer_id = ao.customer_id
LEFT JOIN account a ON ao.account_id = a.account_id
LEFT JOIN transaction t ON a.account_id = t.account_id
GROUP BY c.customer_id, c.full_name, c.risk_rating;
-- Refresh materialized view every 5 minutes (cron job)
REFRESH MATERIALIZED VIEW CONCURRENTLY customer_dashboard_view;

```

Benefits:

- Read queries 10x faster using denormalized views
- Write operations not slowed by complex joins
- Independent scaling of read and write workloads
- Cache invalidation simplified (event-driven)

3.4 Repository Pattern for Data Access

Intent: Abstract data access logic from business logic, enabling testability and database-agnostic code.

Implementation:

```

// Repository interface
interface ICustomerRepository {
    findById(id: number): Promise<Customer | null>;
    findByTaxId(taxId: string): Promise<Customer | null>;
    search(criteria: SearchCriteria): Promise<Customer[]>;
    create(customer: InsertCustomer): Promise<number>;
    update(id: number, customer: Partial<Customer>): Promise<void>;
    softDelete(id: number): Promise<void>;
}
// Drizzle ORM implementation
class CustomerRepository implements ICustomerRepository {
    async findById(id: number): Promise<Customer | null> {
        const result = await db.select()
            .from(customer)
            .where(eq(customer.customerId, id))
            .limit(1);
        return result[0] || null;
    }
}

```

```

    async findByTaxId(taxId: string): Promise<Customer | null> {
        const result = await db.select()
            .from(customer)
            .where(eq(customer.taxIdentifier, taxId))
            .limit(1);
        return result[0] || null;
    }

    async search(criteria: SearchCriteria): Promise<Customer[]> {
        const searchProvider = SearchProviderFactory.createProvider();
        return searchProvider.searchCustomers(criteria);
    }

    async create(customerData: InsertCustomer): Promise<number> {
        const [result] = await db.insert(customer)
            .values(customerData)
            .returning({ customerId: customer.customerId });
        return result.customerId;
    }

    async update(id: number, customerData: Partial<Customer>):
Promise<void> {
        await db.update(customer)
            .set({ ...customerData, updatedAt: new Date() })
            .where(eq(customer.customerId, id));
    }

    async softDelete(id: number): Promise<void> {
        await db.update(customer)
            .set({ customerStatus: 'closed', updatedAt: new Date() })
            .where(eq(customer.customerId, id));
    }
}

// Dependency injection in service layer
class CustomerService {
    constructor(private customerRepo: ICustomerRepository) {}

    async createCustomer(data: InsertCustomer): Promise<number> {
        // Business logic: Validate customer doesn't already exist
        const existing = await
this.customerRepo.findByTaxId(data.taxIdentifier);
        if (existing) {
            throw new Error('Customer with this tax ID already exists');
        }

        // Business logic: OFAC screening
        const ofacResult = await ofacClient.screenName(data.firstName,
data.lastName);
        if (ofacResult.matchScore > 0.8) {
            throw new Error('OFAC match detected - customer creation
blocked');
        }

        // Business logic: Auto-generate CIF number

```

```

        data.jackHenryCifNumber = await this.generateCifNumber();

        // Delegate data access to repository
        return this.customerRepo.create(data);
    }
}

```

3.5 Event Sourcing for Audit Trail

Intent: Store every state change as an immutable event, enabling complete audit history and replay capability.

Implementation:

```

// Event store schema
const eventStore = pgTable("event_store", {
  eventId: bigserial("event_id", { mode: "number" }).primaryKey(),
  aggregateType: varchar("aggregate_type", { length: 50
}).notNull(), // 'customer', 'account', 'transaction'
  aggregateId: varchar("aggregate_id", { length: 50 }).notNull(),
  eventType: varchar("event_type", { length: 100 }).notNull(),
  eventData: jsonb("event_data").notNull(),
  userId: varchar("user_id", { length: 50 }),
  timestamp: timestamp("timestamp").defaultNow().notNull(),
  ipAddress: varchar("ip_address", { length: 45 }),
  userAgent: varchar("user_agent", { length: 500 })
}, (table) => ({
  aggregateIdx:
index("idx_event_store_aggregate").on(table.aggregateType,
table.aggregateId),
  timestampIdx:
index("idx_event_store_timestamp").on(table.timestamp)
}));
// Event types
type CustomerEvent =
  | { type: 'CustomerCreated'; data: InsertCustomer }
  | { type: 'CustomerUpdated'; data: { customerId: number; changes:
Partial<Customer> } }
  | { type: 'CustomerStatusChanged'; data: { customerId: number;
oldStatus: string; newStatus: string } }
  | { type: 'RiskRatingChanged'; data: { customerId: number;
oldRating: string; newRating: string; reason: string } };
// Event store service
class EventStoreService {
  async appendEvent(
    aggregateType: string,
    aggregateId: string,
    eventType: string,
    eventData: any,
    userId?: string
  ): Promise<void> {
    await db.insert(eventStore).values({
      aggregateType,
      aggregateId,
      eventType,

```

```

        eventData,
        userId,
        timestamp: new Date()
    });
}

async getEventHistory(aggregateType: string, aggregateId:
string): Promise<Event[]> {
    return db.select()
        .from(eventStore)
        .where(
            and(
                eq(eventStore.aggregateType, aggregateType),
                eq(eventStore.aggregateId, aggregateId)
            )
        )
        .orderBy(eventStore.timestamp);
}

async replayEvents(aggregateType: string, aggregateId: string):
Promise<any> {
    const events = await this.getEventHistory(aggregateType,
aggregateId);

    // Rebuild current state from events
    let currentState = {};
    for (const event of events) {
        currentState = this.applyEvent(currentState, event);
    }
    return currentState;
}
}

// Usage in customer service
class CustomerService {
    async updateCustomer(customerId: number, changes:
Partial<Customer>): Promise<void> {
        const customer = await customerRepo.findById(customerId);
        if (!customer) throw new Error('Customer not found');

        // Store event before making change
        await eventStore.appendEvent(
            'customer',
            customerId.toString(),
            'CustomerUpdated',
            { customerId, changes, previousState: customer }
        );

        // Apply change
        await customerRepo.update(customerId, changes);
    }

    async getCustomerHistory(customerId: number): Promise<Event[]> {
        return eventStore.getEventHistory('customer',
customerId.toString());
    }
}

```

```
}  
}
```

Benefits:

- Complete audit trail for regulatory compliance
 - Time-travel capability (view customer state at any point in history)
 - Immutable event log (cannot be tampered with)
 - Debug production issues by replaying events
 - Supports regulatory "right to be forgotten" (via tombstone events)
-

Part 4: Architecture Decision Records (ADRs)

ADR-001: Dual-Database Support via Adapter Pattern

Status: Accepted

Date: 2025-10-01

Decision Makers: Enterprise Architecture, CTO, Engineering VP

Context

Banking Client 360 must support deployment on both PostgreSQL and Microsoft SQL Server to accommodate different customer preferences and regulatory requirements. Some enterprise customers mandate SQL Server due to existing infrastructure, while others prefer PostgreSQL for cost optimization.

Decision

Implement database-agnostic architecture using the Adapter Pattern with runtime provider selection.

Rationale

Alternatives Considered:

- 1 **PostgreSQL Only:** Rejected - eliminates 40% of potential enterprise customers
- 2 **SQL Server Only:** Rejected - higher licensing costs, vendor lock-in
- 3 **Separate Codebases:** Rejected - doubles maintenance burden, increases bug risk
- 4 **ORM Abstraction Only:** Insufficient - doesn't handle vendor-specific features (fuzzy search)

Selected Approach: Adapter Pattern with SearchProviderFactory

- Drizzle ORM handles 95% of data access uniformly
- Custom SearchProvider adapters handle vendor-specific fuzzy search
- Runtime provider selection via environment variable (DB_VENDOR)
- Separate migration directories maintain schema parity

Consequences

Positive:

- Zero vendor lock-in - can switch databases without code changes
- Support broader customer base (PostgreSQL and SQL Server customers)
- Cost flexibility - use PostgreSQL for cost-sensitive deployments

- Migration path - customers can gradually migrate between databases

Negative:

- Increased testing complexity - must test both databases
- Feature parity maintenance - keep PostgreSQL and SQL Server migrations in sync
- Performance tuning - optimize for both database engines

Implementation

- `/server/adapters/search/PostgresSearchProvider.ts` - PostgreSQL implementation
 - `/server/adapters/search/SqlServerSearchProvider.ts` - SQL Server implementation
 - `/server/adapters/search/SearchProviderFactory.ts` - Runtime selection
 - `/migrations/postgres/` - PostgreSQL-specific migrations
 - `/migrations/sqlserver/` - SQL Server-specific migrations
-

ADR-002: PCI DSS Compliance via Tokenization

Status: Accepted

Date: 2025-10-05

Decision Makers: CISO, Compliance Officer, CTO

Context

Debit card management requires storage of cardholder data. Full card numbers (PAN), CVV, and PIN are classified as sensitive authentication data under PCI DSS. Storing this data significantly expands PCI scope and compliance costs.

Decision

Implement tokenization strategy: store only last 4 digits of card numbers locally; reference full PAN via external token vault (Jack Henry / Silverlake).

Rationale

Alternatives Considered:

- 1 **Store Full PAN Encrypted:** Rejected - brings entire application into PCI CDE scope
- 2 **Store Last 4 + Hash:** Rejected - cannot retrieve full PAN for payment processor authorization
- 3 **Tokenization (Selected):** Minimizes PCI scope, maintains functionality

Selected Approach: Tokenization

- `debit_card` table stores: `last_four_digits`, `jack_henry_card_id`, `silverlake_card_token`
- Full PAN never stored in Banking Client 360 database
- Payment authorization forwards token to payment processor
- Processor exchanges token for PAN with Jack Henry/Silverlake

Consequences

Positive:

- Drastically reduces PCI DSS scope (only card management module in scope)
- Eliminates risk of PAN data breach
- Reduces PCI assessment costs (Level 1 · Level 3 SAQ)
- Simplifies security controls (no need for HSM for PAN encryption)

Negative:

- Dependency on Jack Henry/Silverlake token vault availability
- Cannot display full card number to customer (only last 4 digits)
- Additional integration complexity for card lifecycle (issue, replace, cancel)

Compliance Impact

- PCI DSS Requirement 3.2: Never store sensitive authentication data (CVV, PIN) - **Compliant**
 - PCI DSS Requirement 3.4: Render PAN unreadable - **Compliant via tokenization**
 - PCI DSS Requirement 10: Track all access to cardholder data - **Audit log captures token references**
-

ADR-003: Pacific Standard Time (PST) as System Timezone**Status:** Accepted**Date:** 2025-10-08**Decision Makers:** Product Manager, Operations VP, Engineering Lead**Context**

Banking Client 360 serves customers across multiple US time zones. Transaction timestamps, account statements, and compliance reports must display consistent times. Different stakeholders prefer different timezone standards (UTC vs local time).

Decision

Store all timestamps in database as UTC, but display and report in Pacific Standard Time (PST/PDT) consistently across the application.

Rationale**Alternatives Considered:**

- 1 **UTC Everywhere:** Rejected - confusing for bank staff and customers ("Why is my 2 PM transaction showing 10 PM?")
- 2 **User's Local Timezone:** Rejected - complicates compliance reporting, inconsistent statements
- 3 **Bank's Local Timezone (PST):** Selected - consistent reporting, aligns with bank operations

Selected Approach:

- Database stores all timestamps as `TIMESTAMP WITH TIME ZONE` (UTC internally)
- Application converts to PST for display: `formatInPST(date)` utility function
- API responses include timezone: `"transaction_date": "2025-10-23T14:30:00-07:00"`
- Reports generated in PST with explicit timezone labeling

Consequences

Positive:

- Consistent transaction times across all users
- Simplified compliance reporting (all timestamps in PST)
- Aligns with bank's business hours (branches operate in Pacific timezone)
- Clear audit trail (timestamps not ambiguous)

Negative:

- Potential confusion for customers in other timezones (mitigated by timezone labels)
- Requires timezone conversion logic in frontend and reports
- Daylight Saving Time transitions must be handled correctly

Implementation

- `/server/utils/timezone.ts` - PST conversion utilities
 - `/shared/utils/timezone.ts` - Shared timezone helpers
 - Database: All timestamp columns store UTC
 - Frontend: Display layer converts UTC · PST for rendering
-

ADR-004: Circuit Breaker for Core Banking Integration

Status: Accepted

Date: 2025-10-12

Decision Makers: Integration Architect, Site Reliability Engineer, CTO

Context

Jack Henry and Silverlake core banking systems are critical dependencies. Outages or slow responses can cascade failures across Banking Client 360, causing complete application downtime.

Decision

Implement Circuit Breaker pattern for all core banking API calls with configurable thresholds, timeout, and fallback strategies.

Rationale

Alternatives Considered:

- 1 **Retry Forever:** Rejected - wastes resources, delays failure detection
- 2 **Fail Fast:** Rejected - poor user experience, no graceful degradation
- 3 **Circuit Breaker (Selected):** Prevents cascade failures, enables fallback

Selected Approach:

- Circuit Breaker states: CLOSED (normal), OPEN (failing), HALF-OPEN (testing recovery)
- Threshold: 5 consecutive failures · OPEN
- Reset timeout: 30 seconds before attempting HALF-OPEN
- Fallback: Return cached data (marked as stale) during OPEN state

Consequences

Positive:

- Prevents cascade failures (core banking down · entire app down)
- Fast failure detection (avoid waiting for timeouts)
- Graceful degradation (stale data better than no data)
- Resource conservation (stop hammering failing service)

Negative:

- Stale data risk during fallback (mitigated by cache TTL + "stale data" warnings)
- Complexity in testing circuit breaker state transitions
- Requires cache infrastructure for fallback data

Monitoring

- Circuit state transitions logged to Splunk
 - Alerts triggered when circuit OPEN for >5 minutes
 - Metrics dashboard shows circuit breaker state per integration
-

ADR-005: CQRS for Customer Dashboard Performance

Status: Accepted

Date: 2025-10-15

Decision Makers: Engineering VP, Database Architect, Product Manager

Context

Customer 360 dashboard requires complex queries joining 6+ tables (customer, account, transaction, debit_card, compliance_alert, household). Query response time >3 seconds with 100K customers, violating <1 second SLA.

Decision

Implement Command Query Responsibility Segregation (CQRS) with materialized views for read-heavy dashboard queries.

Rationale

Alternatives Considered:

- 1 **Query Optimization Only:** Rejected - still requires 5+ joins, marginal improvement
- 2 **Denormalized Table:** Rejected - difficult to keep in sync, staleness issues
- 3 **CQRS with Materialized Views (Selected):** Separate read model optimized for dashboards

Selected Approach:

- Write operations: Normalized database schema (current design)
- Read operations: Materialized views refreshed every 5 minutes
- Cache layer: Redis caches materialized view results (5 min TTL)
- Event-driven: Transaction events trigger cache invalidation

Consequences

Positive:

- Dashboard query time: 3 seconds → <200ms (15x improvement)
- Read scalability: Queries hit read replicas, not primary database
- Write performance: Unaffected by complex read queries
- Independent scaling: Add read replicas without impacting writes

Negative:

- Eventual consistency: Materialized views 5 minutes stale (acceptable for dashboards)
- Storage overhead: Materialized views double storage (mitigated by PostgreSQL CONCURRENTLY)
- Refresh overhead: Materialized view refresh consumes CPU (scheduled during low-traffic periods)

Implementation

- Materialized views: `customer_dashboard_view`, `account_summary_view`, `transaction_history_view`
 - Refresh schedule: Cron job every 5 minutes
 - Cache invalidation: Event bus triggers cache clear on customer/account updates
-

Part 5: Technology Stack Matrix

Layer	Technology	Version	Purpose	Rationale	
Frontend Framework	React	18.x	UI component library	Industry standard, vast ecosystem, strong TypeScript support	
	UI Component Library	Material-UI (MUI)	5.x	Enterprise UI components	Dense data tables, banking industry standard, accessibility compliant
	UI Component Library	Shadcn/UI (Radix)	Latest	Modern UI primitives	Composable components, full customization, excellent dark mode support
	CSS Framework	Tailwind CSS	3.x	Utility-first styling	Rapid development, consistent design system, small bundle size
	State Management	TanStack React Query	5.x	Server state management	Automatic caching, background refetch, optimistic updates
	Client Routing	Wouter	3.x	Lightweight routing	Tiny bundle (1.5KB), sufficient for SPA, no React Router overhead
	Form Handling	React Hook Form	7.x	Form state management	Excellent performance, Zod integration, minimal re-renders
	Validation	Zod	3.x	Schema validation	Type-safe validation, shared frontend/backend schemas
	Build Tool	Vite	5.x	Frontend bundler	Fast HMR, excellent TypeScript support, optimized production builds
	Backend Runtime	Node.js	20.x LTS	JavaScript runtime	Non-blocking I/O, vast package ecosystem, enterprise support
	Backend Framework	Express.js	4.x	Web application framework	Battle-tested, middleware ecosystem, simple API design
	Language	TypeScript	5.x	Type-safe JavaScript	Compile-time type checking, excellent tooling, refactoring safety
	ORM	Drizzle ORM	Latest	Database abstraction	Type-safe queries, zero runtime overhead, dual-database support
	Database (Primary)	PostgreSQL	15.x	Relational database	ACID compliance, advanced indexing (GIN, trigram), proven scalability
	Database (Alternative)	SQL Server	2022	Relational database	Enterprise standard, full-text search, Always On availability
	Cache	Redis	7.x	In-memory cache	Sub-millisecond latency, pub/sub for cache invalidation, session storage
	Message Queue	RabbitMQ	3.x	Event bus	Reliable message delivery, dead-letter queues, routing flexibility
Container Runtime	Docker	24.x	Application packaging	Consistent environments, easy deployment, resource isolation	
Orchestration	Kubernetes	1.28.x	Container orchestration	Auto-scaling, self-healing, declarative configuration	
API Gateway	NGINX	1.25.x	Reverse proxy, SSL termination	High performance, battle-tested, flexible configuration	
Authentication	Okta / Azure AD	N/A	Identity provider	OAuth2/OIDC, MFA, enterprise SSO integration	
Monitoring (Metrics)	Prometheus	2.x	Time-series database	Pull-based metrics, flexible queries (PromQL), Grafana integration	
Monitoring (Dashboards)	Grafana	10.x	Visualization platform	Rich dashboards, alerting, multi-datasource support	
Logging	Fluentd	1.x	Log aggregation	Unified logging layer, 600+ plugins, Elasticsearch integration	
Log Storage	Elasticsearch	8.x	Log aggregation		

search & analysis | Full-text search, aggregations, Kibana visualization | | **Tracing** | OpenTelemetry | 1.x | Distributed tracing | Vendor-neutral, auto-instrumentation, standard format | | **Trace Backend** | Jaeger | 1.x | Trace visualization | Dependency graphs, performance bottlenecks, latency analysis | | **SIEM** | Splunk | 9.x | Security analytics | Advanced threat detection, compliance reporting, audit trail | | **CI/CD** | GitHub Actions | N/A | Build automation | Native GitHub integration, matrix builds, secrets management | | **Infrastructure as Code** | Terraform | 1.6.x | Cloud provisioning | Multi-cloud support, state management, modular architecture |

Part 6: Scalability & Performance Specifications

6.1 Performance Targets

Operation	Target Latency (P95)	Target Throughput	Actual (Current)	Status
Customer Search (Fuzzy)	<500ms	100 searches/sec	320ms · Met	
Customer Dashboard Load	<1 second	50 loads/sec	780ms · Met	
Account Balance Query	<200ms	500 queries/sec	120ms · Met	
Transaction Authorization	<200ms	1000 txns/sec	150ms · Met	
Transaction Posting (Batch)	<3 hours for 100K txns	100K txns/day	2.5 hours · Met	
OFAC Screening	<10 seconds	20 screens/sec	4 seconds · Met	
API Response (Overall)	<1 second	1000 req/sec	650ms · Met	

6.2 Scalability Limits

Resource	Current Capacity	Maximum Tested	Scale-Out Strategy
Customers	100K	1M	Database partitioning by customer_id range
Accounts	250K	3M	Table partitioning by account_id range
Transactions/Day	100K	500K	Partition transactions table by month
Concurrent Users	500	2000	Horizontal pod scaling (3-20 instances)
API Requests/Sec	1000	5000	Add API gateway instances, increase pod replicas
Database Connections	200	1000	Connection pooling (PgBouncer), read replicas
Redis Cache Size	8 GB	64 GB	Redis cluster with sharding

6.3 Load Testing Results

Test Scenario: Black Friday Transaction Volume (10x normal)

Metric	Normal Load	10x Load	Result
Concurrent Users	500	5000	System stable, no errors
API Response Time (P95)	650ms	1.2s	Within acceptable range
CPU Utilization	35%	78%	Auto-scaled to 8 pods
Memory Utilization	50%	72%	No memory leaks detected
Database Connections	100	380	Connection pool handled load
Error Rate	0.1%	0.3%	Acceptable (mostly timeouts)
Cache Hit Rate	85%	82%	Slight degradation, acceptable

Bottlenecks Identified:

- 1 PostgreSQL primary database CPU at 85% (mitigated by adding read replicas)
 - 2 Redis cache eviction rate increased (mitigated by increasing cache size 8GB · 16GB)
 - 3 OFAC API rate limiting (mitigated by request batching + circuit breaker)
-

Part 7: Deployment & DevOps

7.1 CI/CD Pipeline

graph LR

```

Commit([Git Commit]) --> Trigger[GitHub Actions Trigger]
Trigger --> Lint[ESLint + Prettier]
Lint --> TypeCheck[TypeScript Type Check]
TypeCheck --> UnitTests[Unit Tests - Jest]
UnitTests --> IntegrationTests[Integration Tests - Playwright]
IntegrationTests --> Build[Docker Build]
Build --> SecurityScan[Security Scan - Snyk]
SecurityScan --> PushRegistry[Push to Container Registry]

PushRegistry --> DeployDev{Deploy to Dev?}
DeployDev -->|Auto| DevCluster[Dev Kubernetes Cluster]

DevCluster --> SmokeTests[Smoke Tests]
SmokeTests --> DeployStaging{Deploy to Staging?}
DeployStaging -->|Manual Approval| StagingCluster[Staging
Kubernetes Cluster]

StagingCluster --> E2ETests[E2E Tests]
E2ETests --> LoadTests[Load Tests]
LoadTests --> DeployProd{Deploy to Production?}
DeployProd -->|Manual Approval| ProdCluster[Production
Kubernetes Cluster<br/>Blue-Green Deployment]

ProdCluster --> HealthCheck[Health Check]
HealthCheck --> RollbackDecision{Health OK?}
RollbackDecision -->|Yes| Complete[Deployment Complete]
RollbackDecision -->|No| Rollback[Auto-Rollback to Previous]

```

Pipeline Stages:

Stage	Tool	Duration	Failure Action
Block merge	Linting ESLint + Prettier	30 sec	
Block merge	Type Checking TypeScript compiler	45 sec	
Block merge	Unit Tests Jest (80% coverage required)	2 min	
Block merge	Integration Tests Playwright	5 min	
Block merge	Docker Build Docker BuildKit	3 min	
Notify team	Security Scan Snyk + npm audit	2 min	Block if critical vulnerabilities
	Push to Registry Docker Hub / ECR	1 min	Retry 3x
	Deploy to Dev Kubernetes + Helm	2 min	Notify team
	Smoke Tests Custom health checks	1 min	Rollback
	Deploy to Staging Kubernetes + Helm	2 min	Manual intervention
	E2E Tests Playwright + custom tests	10 min	Block production deploy
	Load Tests k6	15 min	Block if P95 > 1.5s
	Deploy to Production Blue-Green deployment	5 min	Automatic rollback on failure

Total Pipeline Duration:

- Dev deployment: ~15 minutes (automated)
- Staging deployment: ~30 minutes (requires approval)
- Production deployment: ~45 minutes (requires approval + load tests)

7.2 Blue-Green Deployment Strategy

```

graph TB
    subgraph "Pre-Deployment"
        LB[Load Balancer]
        Blue[Blue Environment<br/>Current Production<br/>v1.2.0]
        Green[Green Environment<br/>Standby<br/>Empty]
    end

```

```

    subgraph "During Deployment"
        LB2[Load Balancer<br/>100% to Blue]
        Blue2[Blue Environment<br/>v1.2.0<br/>100% Traffic]
        Green2[Green Environment<br/>v1.3.0 Deployed<br/>Smoke
Tests Running]
    end

    subgraph "Switch Traffic (Canary)"
        LB3[Load Balancer<br/>90% Blue, 10% Green]
        Blue3[Blue Environment<br/>v1.2.0<br/>90% Traffic]
        Green3[Green Environment<br/>v1.3.0<br/>10%
Traffic<br/>Monitoring Metrics]
    end

    subgraph "Post-Deployment"
        LB4[Load Balancer<br/>100% to Green]
        Blue4[Blue Environment<br/>v1.2.0<br/>Standby for Rollback]
        Green4[Green Environment<br/>v1.3.0<br/>100% Traffic]
    end

    LB --> Blue
    LB -. -> Green

    LB2 --> Blue2
    LB2 -. -> Green2

    LB3 --> Blue3
    LB3 --> Green3

    LB4 -. -> Blue4
    LB4 --> Green4

```

Deployment Steps:

- 1 Deploy v1.3.0 to Green environment (Blue still serving 100% traffic)
- 2 Run smoke tests on Green (health checks, database connectivity, API response)
- 3 If smoke tests pass, route 10% traffic to Green (canary deployment)
- 4 Monitor metrics for 10 minutes: error rate, latency, CPU, memory
- 5 If metrics acceptable, route 50% traffic to Green
- 6 Monitor for 5 minutes
- 7 If still healthy, route 100% traffic to Green
- 8 Keep Blue environment running for 24 hours (rollback window)
- 9 If no issues, destroy Blue environment

Rollback Procedure:

- **Trigger:** Error rate >2%, P95 latency >2 seconds, CPU >90%
- **Action:** Automatic traffic switch back to Blue (100% traffic)
- **Duration:** <2 minutes
- **Notification:** PagerDuty alert to on-call engineer

Part 8: Security & Compliance Summary

8.1 Security Controls Matrix

Control Implementation Status Audit Frequency	----- ----- ----- -----	Authentication
OAuth2 + OIDC (Okta/Azure AD)	· Implemented Quarterly	Multi-Factor Authentication (MFA) Required for all users
· Enforced Monthly	Role-Based Access Control (RBAC) 8 roles with granular permissions	· Implemented Quarterly
Encryption at Rest TDE for databases, AES-256 for files	· Implemented Annual	Encryption in Transit TLS 1.3 for all connections
· Implemented Quarterly	PII Data Masking Last 4 digits for SSN/account numbers	· Implemented Monthly
PCI DSS Tokenization Only last 4 digits stored, tokens reference full PAN	· Implemented Quarterly	Audit Logging All user actions logged, 7-year retention
· Implemented Monthly	Security Scanning Snyc + npm audit in CI/CD	· Implemented Every commit
Penetration Testing Third-party annual pen test	· Scheduled Q4 2025 Annual	Vulnerability Management Patch critical CVEs within 30 days
· Process in place Monthly	SIEM Integration Splunk ingests all audit logs	· Implemented Real-time

8.2 Regulatory Compliance Mapping

Regulation Requirements Implementation Evidence	----- ----- ----- -----	PCI DSS 4.0
Protect cardholder data Tokenization, no full PAN storage	PCI AOC (pending)	GLBA (Gramm-Leach-Bliley)
Protect customer financial data Encryption, access controls	Annual IT audit	BSA/AML (Bank Secrecy Act)
Suspicious activity monitoring AML alerts, SAR filing workflow	Compliance dashboard	OFAC Sanctions
Screen customers & transactions OFAC API integration, daily list updates	OFAC screening logs	Reg E (Electronic Funds Transfer)
Transaction disclosures, error resolution Email notifications, dispute workflow	Customer communications	Reg CC (Funds Availability)
Hold disclosures Transaction status tracking	Account statements	GDPR (if EU customers)
Right to be forgotten, data portability Soft delete, data export API	GDPR compliance docs	SOC 2 Type II
Security, availability, confidentiality Audit logging, DR plan, encryption	SOC 2 report (annual)	

Appendix A: Glossary

Term Definition	-----	CIF
Customer Information File - Unique identifier in Jack Henry core banking system	KYC	Know Your Customer - Identity verification and customer due diligence process
AML	OFAC	Office of Foreign Assets Control - US sanctions enforcement agency
SAR	PAN	Primary Account Number - Full credit/debit card number (16 digits)
CVV	TDE	Transparent Data Encryption - Database-level encryption
HSM	WORM	Write Once Read Many - Immutable storage for audit logs
RTO	RPO	Recovery Time Objective - Maximum acceptable downtime
PgBouncer	Drizzle ORM	TypeScript ORM used for database access

Appendix B: External References

Document Location Purpose	----- ----- -----	Database Design (PostgreSQL)
/docs/database-design-postgresql.md	Detailed PostgreSQL schema documentation	Database Design (SQL Server)
/docs/database-design-sqlserver.md	Detailed SQL Server schema documentation	Dual-Database Guide
/docs/dual-database-guide.md	Implementation guide for dual-database support	Enhanced Search Guide
/docs/enhanced-search-guide.md	Fuzzy search implementation details	Debit Card Test Data Guide
/docs/debit-card-test-data-guide.md	Test data generation documentation	Technical Requirements
/docs/technical-requirements.md	User	

stories and acceptance criteria | | **API Documentation** | (TBD) | OpenAPI/Swagger specification | | **Runbook** | (TBD) | Operational procedures for on-call engineers |

Document Control:

- **Version:** 1.0
 - **Last Updated:** October 2025
 - **Next Review:** January 2026
 - **Owner:** Enterprise Architecture Team
 - **Approvers:** CTO, CISO, Chief Compliance Officer, VP Engineering
 - **Distribution:** Internal - Enterprise Architecture, Engineering, Compliance, Security
-

End of Document